CloudStore

Full Operator & Developer Guide

How to bring it back up next time, how every part of the system works, and how to troubleshoot common problems.

Document	Full Operator & Developer Guide
Version	1.0
Date	May 16, 2026
Audience	Anyone running or modifying CloudStore
Companion	CloudStore_Documentation.pdf (concise design spec)

Contents

1. Quick start (next time you sit down)

Three scenarios, three sets of commands. Pick the one that matches where you want the app to run.

1a. Just on Windows (dev)

```
cd "C:\Users\dell\Desktop\VCC Project"
python run.py
# open http://127.0.0.1:5000
```

1b. On the Ubuntu VM (already provisioned)

```
# 1. Start the VM in VirtualBox
# 2. Log in as nehad
cd ~/cloudstore
source .venv/bin/activate
python run.py
# on the Windows host browser, open:
#   http://192.168.100.73:5000
# (if the VM IP changed, run `ip a | grep inet` first)
```

1c. After you change code on Windows, push it to the VM

```
scp -r "C:\Users\dell\Desktop\VCC Project\app" nehad@192.168.100.73:~/cloudstore/
# the Flask dev server on the VM auto-reloads on file change
```

Note: the VM IP is assigned by your home router via DHCP. If you reboot the VM or your router, the IP may change. Always check with `ip a | grep inet` on the VM.

2. What CloudStore is

A small web application for personal file storage. Users sign up, log in, and upload files into a private bucket. Admins have a separate dashboard where they can see everything, promote / demote users, and delete content.

It runs on Flask. The database is SQLite. File bytes live on the host's filesystem; only metadata (name, size, owner, category, timestamp) is in the database. The system is designed to be deployed to a Linux VM that your Windows host can reach over the LAN.

3. Network topology

Your Windows machine and the Ubuntu VM each have their own IP on the same home network. The VM's network adapter is set to **Bridged**, which means VirtualBox places it on the LAN directly rather than hiding it behind NAT.

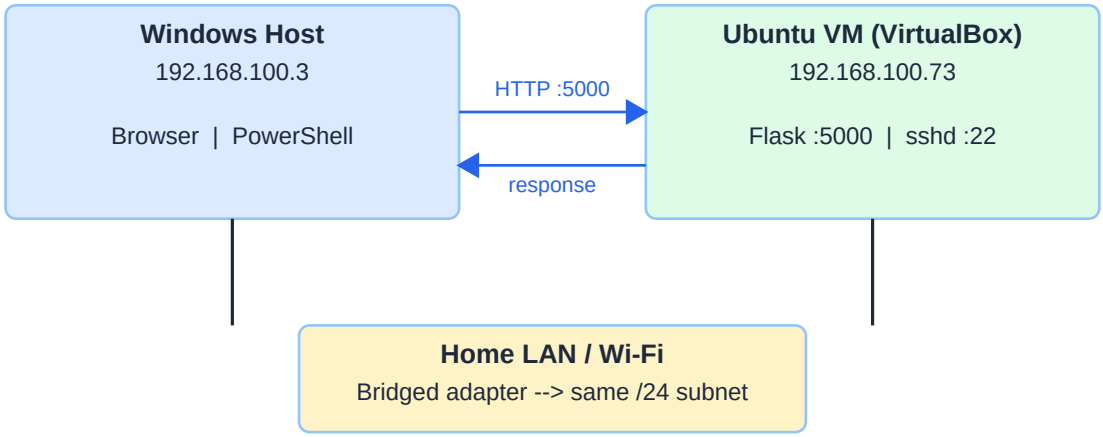


Figure 1 - Network topology (Windows host <-> VM over bridged adapter)

- Host IP: `192.168.100.3` (your Windows machine)
- VM IP: `192.168.100.73` (Ubuntu, bridged)
- App port: `5000` (Flask dev server, listens on 0.0.0.0)
- SSH port: `22` (used by `scp` for file transfer)

4. System architecture

Top-to-bottom view of one request. The browser talks to Flask; Flask uses its extensions to read/write the database and the filesystem.

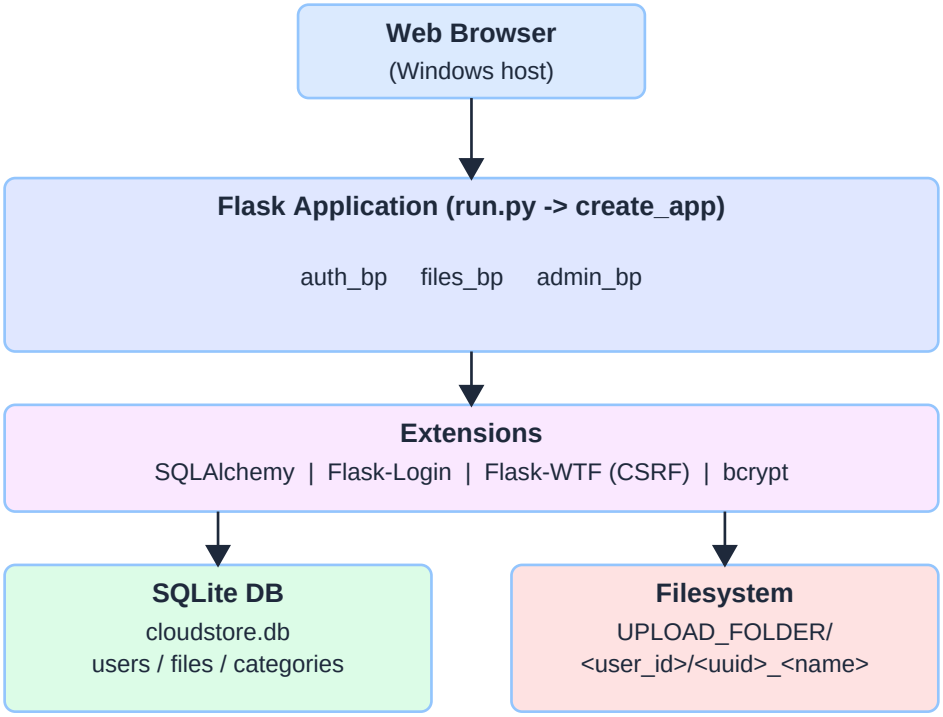


Figure 2 - Layered architecture

- **Browser** - renders HTML, sends form posts with a CSRF token.
- **Flask app** - dispatches the URL to one of three blueprints (auth, files, admin).
- **Extensions** - SQLAlchemy for the DB, Flask-Login for sessions, Flask-WTF for CSRF, bcrypt for passwords.
- **SQLite DB** - rows for users / files / categories.
- **Filesystem** - the raw bytes of every uploaded file, namespaced per user.

5. Code components

The codebase is organised by feature. Each blueprint owns its routes and (where useful) its forms. Shared concerns sit in `services/` and `utils/`.

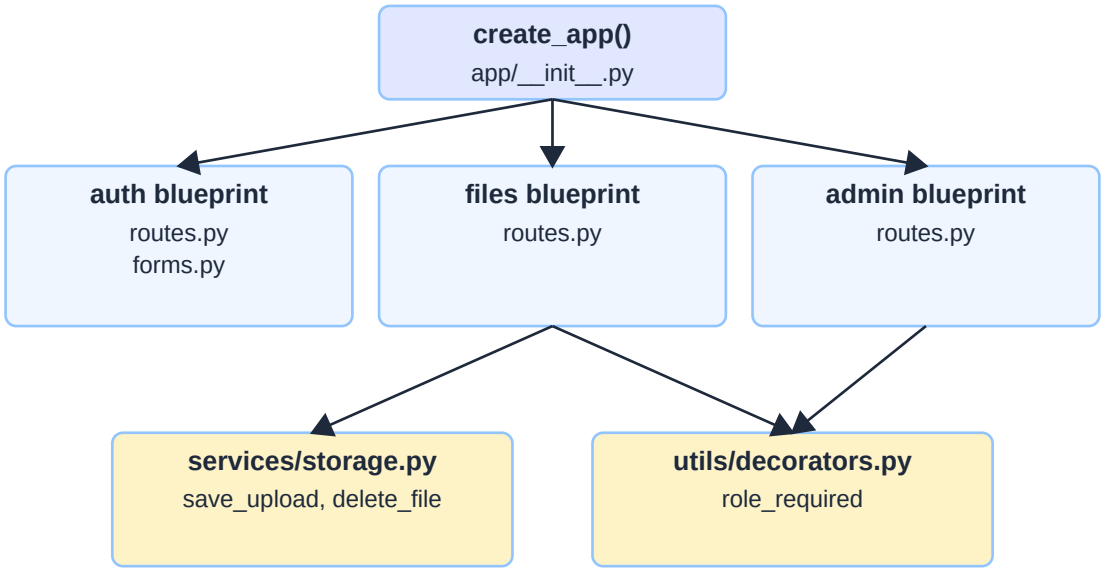


Figure 3 - Component diagram

```
app/
  __init__.py       create_app(), CLI commands
  extensions.py     db, csrf, login_manager singletons
  models.py         User, File, Category
  auth/             signup / login / logout
  files/            upload / download / delete (user scope)
  admin/            list users, change role, delete user, list files, categories
  services/storage.py   save_upload, delete_file
  utils/decorators.py   role_required('admin')
  templates/        Jinja2 (base + auth + files + admin)
  static/style.css  single stylesheet
```

6. Data model (ER diagram)

Three tables. A user owns many files. A file optionally belongs to a category. Categories are independent of users.

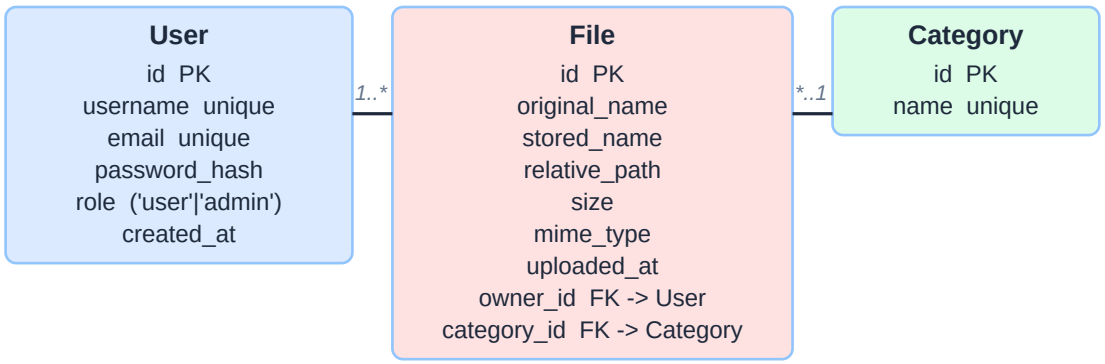


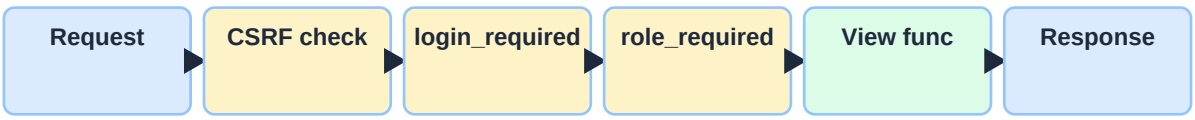
Figure 4 - Entity-relationship diagram

Key fields explained:

- **User.role** - `'user'` or `'admin'`. All role checks read this column.
- **User.password_hash** - bcrypt hash with per-user salt. Plain text never touches the DB.
- **File.relative_path** - `<user_id>/<uuid>_<name>`. Stored relative to `UPLOAD_FOLDER` so the same DB row works on Windows dev and on the VM under `/var/cloudstore/uploads`.
- **File.owner_id** - the foreign key every ownership check reads.

7. Request lifecycle

Every request passes through a series of gates before reaching the view function. If any gate fails, the request short-circuits with a 4xx and no business logic runs.



Failure at any gate -> early 4xx response (no view executed)

Figure 5 - Order of checks

Gate	What it does	Failure
CSRF check	Flask-WTF verifies the form's csrf_token on every POST.	400 Bad Request
login_required	Flask-Login checks the session cookie.	302 -> /auth/login?next=...
role_required	Custom decorator checks current_user.role.	403 Forbidden
Ownership check	Per-route: file.owner_id == current_user.id (or admin).	403 Forbidden
View function	Runs the actual business logic.	—

8. Login flow (sequence)

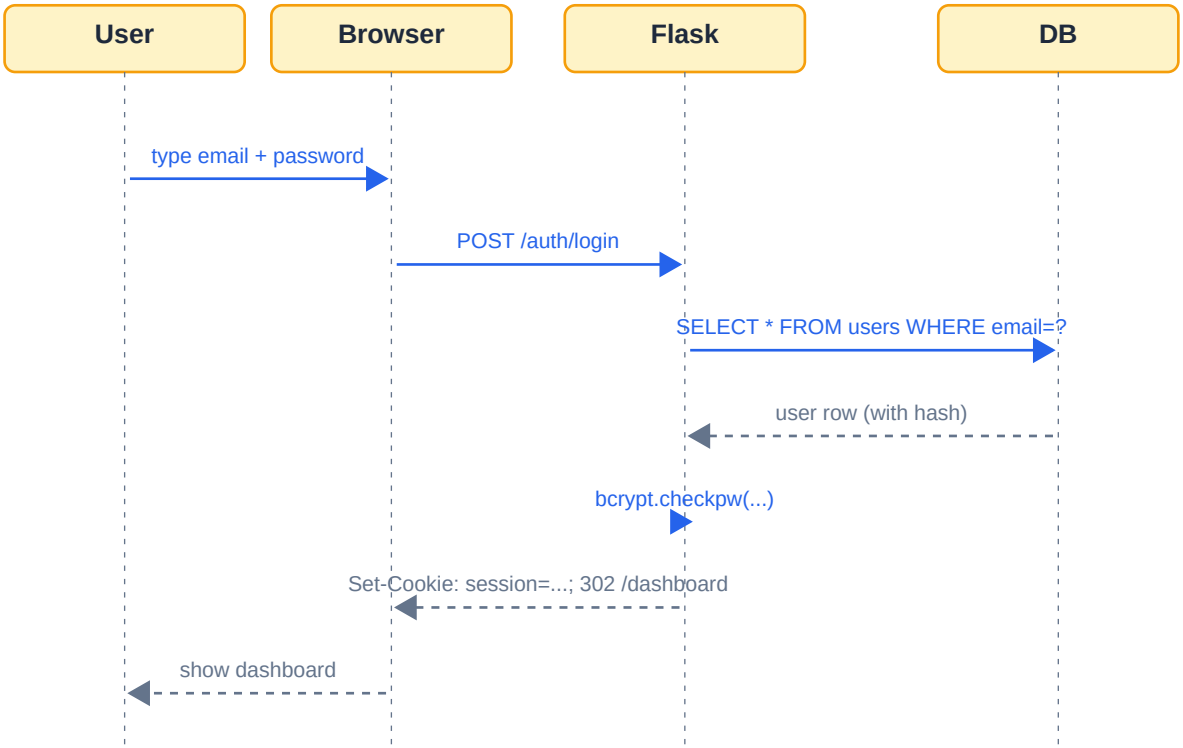


Figure 6 - Login sequence

The password is hashed once at signup with `bcrypt.hashpw`. At login time, `bcrypt.checkpw` hashes the submitted password with the stored salt and compares in constant time. On success, Flask-Login signs a session cookie and the browser is redirected to `/dashboard`.

9. Upload flow (sequence)

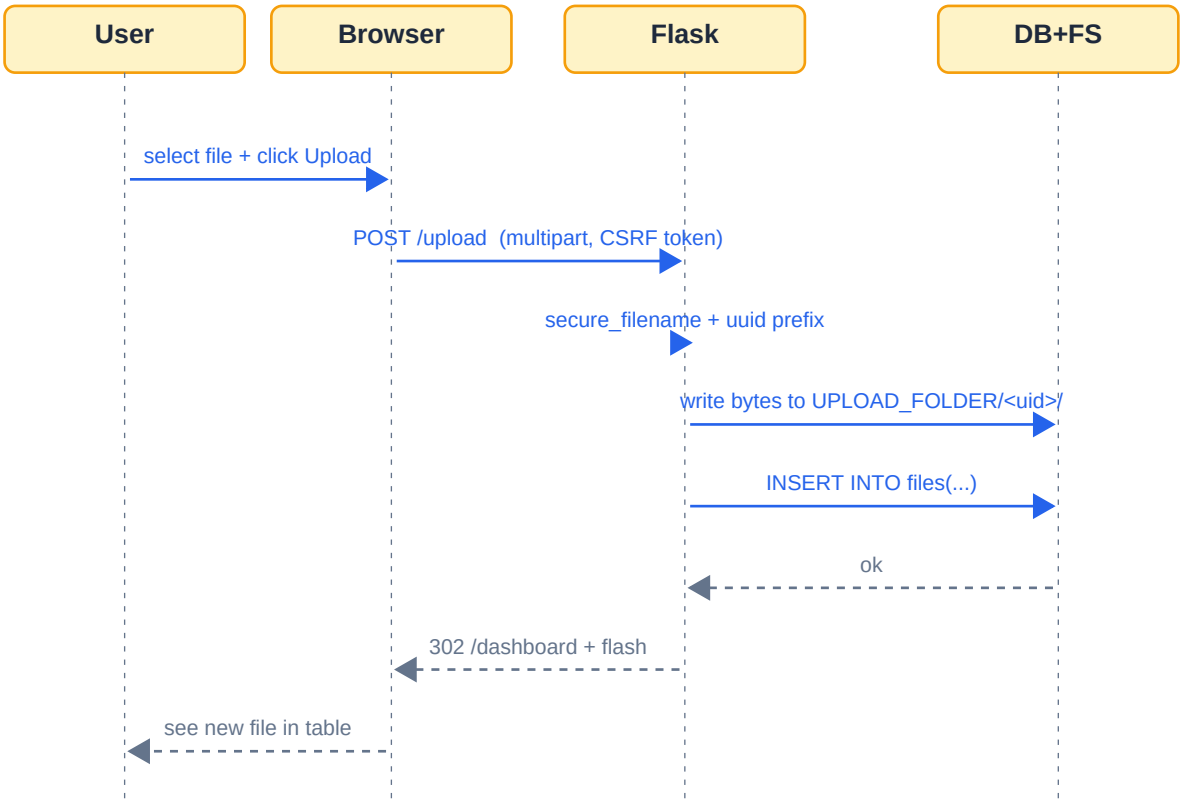


Figure 7 - Upload sequence

Two security steps before the file ever touches disk:

- **secure_filename** from werkzeug strips path separators, control characters, and trailing dots — so `../../etc/passwd` becomes `etc_passwd`.
- A **UUID hex prefix** is prepended so two users (or one user twice) uploading `report.pdf` never collide and the stored name is unguessable.

10. Download flow (sequence with auth check)

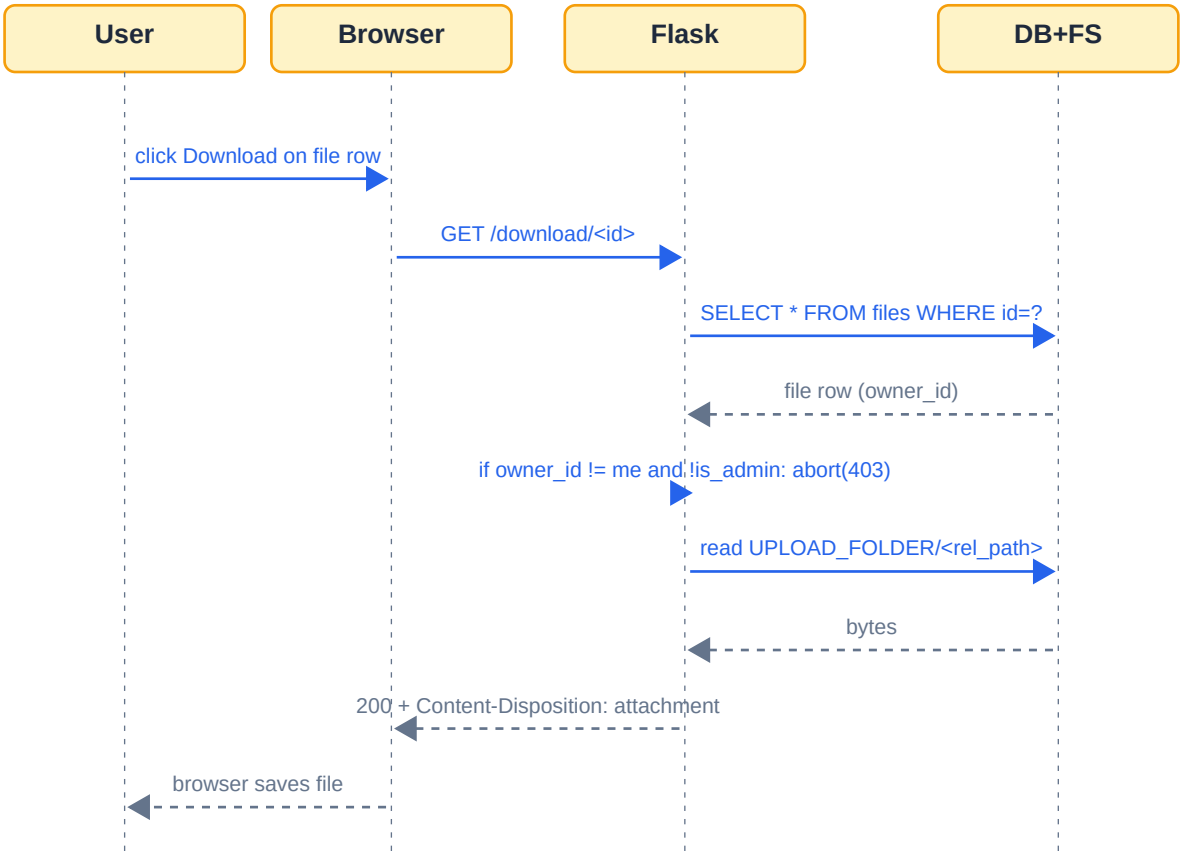


Figure 8 - Download sequence (the highlighted ownership check is the security boundary)

Even if a user guesses another user's file id and types it into the URL bar, the ownership check returns 403 before any bytes are read.

```
# app/files/routes.py
def _get_owned_file_or_404(file_id):
    file = db.session.get(File, file_id)
    if file is None:
        abort(404)
    if file.owner_id != current_user.id and not current_user.is_admin:
        abort(403)
    return file
```

11. Admin operations

Action	Route	Notes
See system stats	GET /admin/	user count, file count, storage used
List all users	GET /admin/users	shows role + file count per user
Promote / demote	POST /admin/users/<id>/role	cannot change own role
Delete user (and files)	POST /admin/users/<id>/delete	cascade deletes their files on disk too

List every file	GET /admin/files	shows owner column
Delete any file	POST /delete/<id>	ownership check passes because is_admin
Manage categories	GET/POST /admin/categories	add new tags users can apply at upload

12. Day-2 ops (common tasks)

Create a new admin from the CLI

```
# on the VM, with venv activated
export FLASK_APP=run.py
flask create-admin alice alice@example.com 'StrongPassword!'
```

Reset the database (wipe everything)

```
rm cloudstore.db
rm -rf uploads/*
flask init-db
flask create-admin admin admin@example.com 'ChangeMe123!'
```

Back up the data

```
tar -czf cloudstore-backup-$(date +%F).tar.gz cloudstore.db uploads/
```

Pull the project off the VM (e.g. to share)

```
# from Windows PowerShell
scp -r nehad@192.168.100.73:~/cloudstore C:\Users\dell\Desktop\cloudstore-from-vm
```

13. Troubleshooting

Symptom	Likely cause	Fix
Browser can't reach VM IP	Adapter is NAT, not Bridged	VirtualBox -> VM Settings -> Network -> Bridged Adapter
SSH 'connection refused'	openssh-server not installed on VM	sudo apt install -y openssh-server && sudo systemctl ena
apt: 'Temporary failure resolving'	DNS broken on VM	echo 'nameserver 8.8.8.8' | sudo tee /etc/resolv.conf
'csrf_token' is undefined in templates	CSRFProtect not initialised	Already fixed - app/__init__.py calls csrf.init_app(app)
403 on /admin/	User role is 'user', not 'admin'	flask create-admin <username> ... or promote via UI
File 413 too large	Upload exceeds MAX_CONTENT_LENGTH	Raise MAX_CONTENT_LENGTH in .env, restart Flask
VM IP changed	DHCP lease rotated	Run 'ip a | grep inet' on the VM, update the URL
Server still running after Ctrl+C didn't kill it	Background process from earlier session	On Linux: pkill -f 'python run.py' - on Windows: stop the P

14. Security checklist

Tick these off before showing CloudStore to anyone outside your home network. The dev server is fine for a classroom or demo; the items marked **production** matter only if you put this on the public internet.

- **SECRET_KEY** is a long random string in `.env`, not the placeholder.
- Default admin password (`ChangeMe123!`) has been changed.
- `.env` is in `.gitignore` (it is).
- Uploaded files are saved per-user with UUID prefix (they are).
- Every download / delete route runs the ownership check (it does).
- CSRF protection is global (CSRFProtect.init_app, yes).
- **(production)** Use Gunicorn + Nginx, not the Flask dev server.
- **(production)** Serve over HTTPS with a real certificate.
- **(production)** Switch DB to MySQL or Postgres; back up nightly.
- **(production)** Add rate limiting on /auth/login to slow down brute force.

15. Production-style stack (Gunicorn + Nginx)

When you are ready to graduate off the Flask dev server, the recommended stack is Gunicorn as the WSGI server and Nginx as the reverse proxy on port 80. This survives a terminal closing and serves static files faster.

Install + run Gunicorn

```
source .venv/bin/activate
pip install gunicorn  # already in requirements.txt
gunicorn -w 4 -b 127.0.0.1:8000 run:app
```

Make it a systemd service

```
sudo tee /etc/systemd/system/cloudstore.service > /dev/null <<'EOF'
[Unit]
Description=CloudStore (Gunicorn)
After=network.target

[Service]
User=nehad
WorkingDirectory=/home/nehad/cloudstore
Environment="PATH=/home/nehad/cloudstore/.venv/bin"
ExecStart=/home/nehad/cloudstore/.venv/bin/gunicorn -w 4 -b 127.0.0.1:8000 run:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now cloudstore
```

Front it with Nginx

```
sudo apt install -y nginx
sudo tee /etc/nginx/sites-available/cloudstore > /dev/null <<'EOF'
server {
  listen 80;
  server_name 192.168.100.73;
  client_max_body_size 100M;
  location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }
}
EOF
sudo ln -sf /etc/nginx/sites-available/cloudstore /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
sudo ufw allow 80
```

Now open `http://192.168.100.73` from Windows (no `:5000` any more).

16. FAQ

Q. Why two PDFs?

The other PDF (CloudStore_Documentation.pdf) is the design spec — concise, good for handing to a reviewer. This one is the operational + architectural deep dive with diagrams.

Q. Can I run only on Windows and skip the VM?

Yes. `python run.py` on Windows is fully functional. The VM exists to mirror a realistic deployment (Linux filesystem layout, real WSGI server, separate machine over the network) and to satisfy the project brief.

Q. The VM rebooted and the IP changed. Now what?

Log in to the VM, run `ip a | grep inet`, and use the new `192.168.x.x` address in the browser. If you want a fixed IP, reserve one in your router's DHCP settings.

Q. How big can an uploaded file be?

The default cap is 100 MB, set by `MAX_CONTENT_LENGTH` in `.env`. Increase it there and restart Flask. If you go above ~500 MB, also raise `client_max_body_size` in Nginx.

Q. How do I add a new feature?

Add a route to the most relevant blueprint, put any non-trivial logic in `services/`, add a template under `app/templates/`, and link it from `base.html`. Use `@login_required` and (for admin features) `@role_required('admin')`.

Generated by `docs/generate_full_pdf.py`. *Re-run that script whenever the codebase changes.*